A Deep Learning–Driven Digital Twin for Quality Prediction and Process Optimisation in CNC Manufacturing

Student Name:

Student ID:

Abstract

The paper provides an all-embracing machine learning model for defect risk prediction in CNC milling processes based on the Tool Wear Detection in CNC Mill dataset. Tabular process parameters and multivariate time-series sensor signals are used to represent machining dynamics. Several deep learning networks, such as Multilayer Perceptron (MLP), Long Short-Term Memory (LSTM), Convolutional Neural Network (CNN), and a hybrid CNN-Long Short-Term Memory (LSTM), are created and discussed. SHAP analysis is used in obtaining model interpretability, whereas Bayesian optimization is used in providing optimal machining parameters. The assessment of the models based on the experiment results illustrates the relative merits of each model and suggests the appropriateness of deep learning-based digital twins for predictive quality assurance of manufacturing.

Table of Contents

Abstract 2

1. Introduction.....5

2. Related Work.....6

2.1 Machine Learning for Tool Wear Monitoring 6

2.2 Deep Learning for Time-Series Sensor Analysis 6

2.3 Hybrid and Multimodal Learning Approaches.....7

2.4 Explainable AI and Process Optimization 7

3. Methodology 7

3.1 Dataset and Preprocessing 8

3.2 Model Development 9

3.3 Training and Implementation Details 10

3.4 Evaluation Metrics.....14

4. Experiments and Results.....15

4.1 MLP Results 15

4.2 LSTM Results 16

4.3 CNN Results.....16

4.4 CNN–LSTM Results 17

4.5 SHAP Explainability Results 17

5. Discussion 18

6. Conclusion and Future Work 20

References 21

Appendix 23

List of Figures

Figure 1 Dataset Overview 8
Figure 2 Cleaned Data overview 9
Figure 3 Data Preprocessing part 10
Figure 4 Data Preprocessing outcome 11
Figure 5 Time Series Preprocessing 11
Figure 6 LSTM Model Development 12
Figure 7 CNN Model Development 13
Figure 8 CNN LSTM Hybrid Model 13
Figure 9 MLP Implementation 14
Figure 10 SHAP Implementation 14
Figure 11 MLP Result 15
Figure 12 LSTM Result 16
Figure 13 CNN Result 17
Figure 14 CNN–LSTM Results 17
Figure 15 Shap Result 18
Figure 16 All The evaluation visual results 19

1. Introduction

The wear of the tools, the process parameters and dynamic sensor behaviour constitute the key aspects of quality control in the manufacturing process of CNC machining environment. Unnoticed reduction in tool life may lead to flawed parts, higher scrap, planned unproductive time and higher operation expenses. The rigidity of traditional rule-based surveillance and threshold-based instruments is not able to support the complicated, nonlinear interactions of contemporary high- Hereto machining operations. The current research using machine learning and deep learning has made possible the possibility of data-based modelling techniques that can learn defect trends directly using historical process data and high-rate sensor signals. The project suggests a digital twin structure based on the approach of deep learning as a predictor of defect risk when CNC milling is being performed [1]. The framework will help to enhance predictive accuracy, develop better interpretability with explainable AI methods, and assist in making intelligent decisions and optimisation in smart manufacturing systems by solving the problem of integrating sensor data in time-series with tabular process parameters.

Aim

This project will focus on the creation of an explainable digital twin of deep learning to predict the risk of defects and optimise the processes in the CNC machining tasks.

Objectives

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

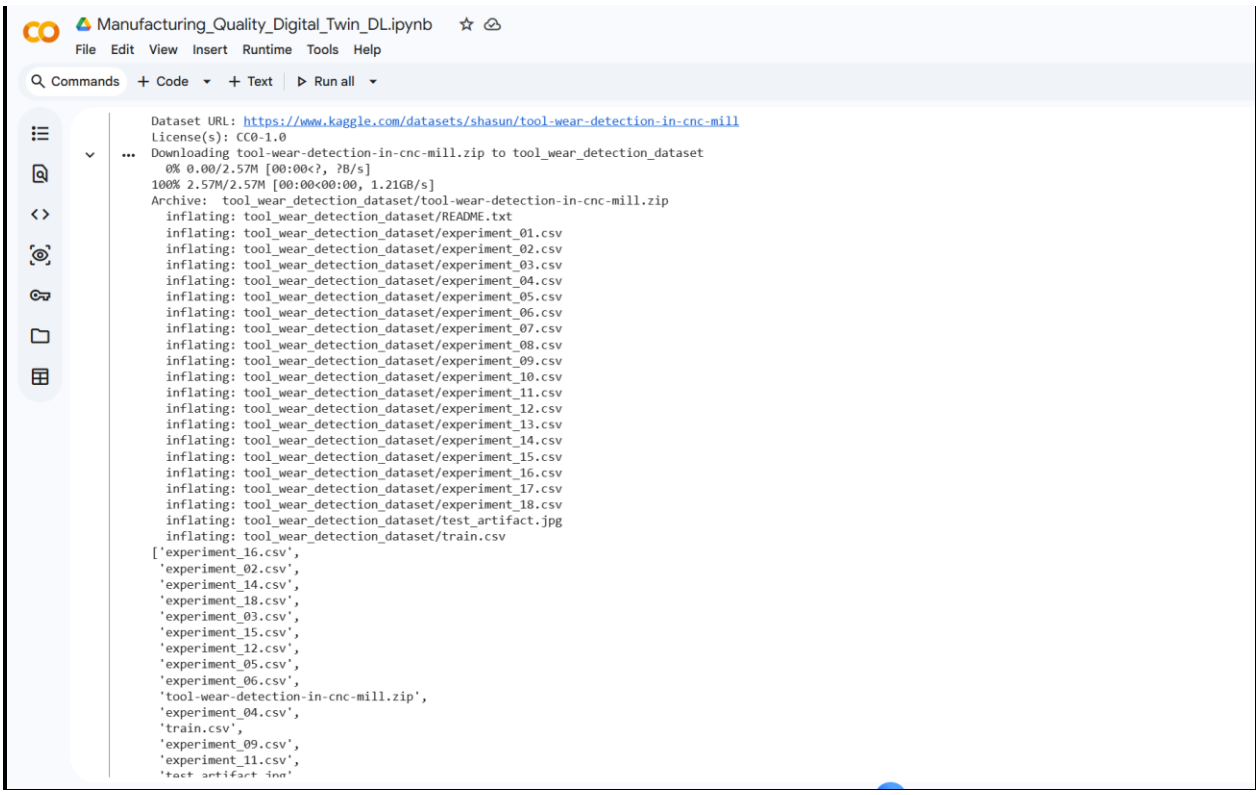
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

```python
print(train_df["defect_risk"].value_counts())
```

```
===== CLEANED TRAIN DATA =====
    no material  feedrate  clamp_pressure  tool_condition  \
0    1      wax         6             4.0               0
1    2      wax        20             4.0               0
2    3      wax         6             3.0               0
3    4      wax         6             2.5               0
4    5      wax        20             3.0               0
5    6      wax         6             4.0               1
6    7      wax        20             4.0               1
7    8      wax        20             4.0               1
8    9      wax        15             4.0               1
9   10      wax        12             4.0               1
10  11      wax         3             4.0               0
11  12      wax         3             3.0               0
12  13      wax         3             4.0               1
13  14      wax         3             3.0               1
14  15      wax         6             3.0               1
15  16      wax        20             3.0               1
16  17      wax         3             2.5               0
17  18      wax         3             2.5               1

    machining_finalized  passed_visual_inspection  defect_risk
0                     1                       1.0            0
1                     1                       1.0            0
2                     1                       1.0            0
3                     0                       0.0            1
4                     0                       0.0            1
5                     1                       0.0            1
6                     0                       0.0            1
7                     1                       0.0            1
8                     1                       0.0            1
9                     1                       0.0            1
10                    1                       1.0            0
11                    1                       1.0            0
12                    1                       1.0            1
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
import numpy as np
from sklearn.preprocessing import StandardScaler


# Select tabular features

TABULAR_FEATURES = ['feedrate', 'clamp_pressure']
TARGET = 'defect_risk'

X_tabular = train_df[TABULAR_FEATURES].values
y = train_df[TARGET].values


# Normalize tabular features

scaler = StandardScaler()
X_tabular_scaled = scaler.fit_transform(X_tabular)

print("===== TABULAR INPUT SHAPE =====")
print(X_tabular_scaled.shape)

print("\n===== TARGET SHAPE =====")
print(y.shape)


# Create experiment-to-label mapping

experiment_labels = {
    row['no']: row['defect_risk']
    for _, row in train_df.iterrows()
}

print("\n===== EXPERIMENT → LABEL MAPPING (Sample) =====")
for k in list(experiment_labels.keys())[:5]:
    print(f"Experiment {k} → Defect Risk: {experiment_labels[k]}")
```

[redacted]

```
===== TABULAR INPUT SHAPE =====
(18, 2)

===== TARGET SHAPE =====
(18,)

===== EXPERIMENT → LABEL MAPPING (Sample) =====
Experiment 1 → Defect Risk: 0
Experiment 2 → Defect Risk: 0
Experiment 3 → Defect Risk: 0
Experiment 4 → Defect Risk: 1
Experiment 5 → Defect Risk: 1
```

[redacted]

```
# Time-series preprocessing
sensor_columns = [
    col for col in sample_ts.columns
    if col not in ['Machining_Process']
]

scaler_ts = StandardScaler()

X_seq = []
y_seq = []
tab_seq = []

for idx in range(1, 19):
    ts_file = f"experiment_{idx:02d}.csv"
    ts_path = f"{DATA_PATH}/{ts_file}"

    ts_df = pd.read_csv(ts_path)
    ts_data = scaler_ts.fit_transform(ts_df[sensor_columns].values)

    for start in range(0, len(ts_data) - SEQUENCE_LENGTH, STRIDE):
        end = start + SEQUENCE_LENGTH
        X_seq.append(ts_data[start:end])
        y_seq.append(y[idx - 1])
        tab_seq.append(X_tab_scaled[idx - 1])

X_seq = np.array(X_seq)
tab_seq = np.array(tab_seq)
y_seq = np.array(y_seq)

print("\n===== SEQUENCE DATA SHAPE =====")
print("X_seq:", X_seq.shape)
print("Tabular:", tab_seq.shape)
print("Target:", y_seq.shape)
```

```
===== TABULAR DATA NORMALIZED =====
[[-0.52593118  0.96152395]
 [ 1.45219805  0.96152395]
 [-0.52593118 -0.68680282]
```

[redacted]

LSTM MODEL (PyTorch) – DEFECT / TOOL-WEAR PREDICTION

```python
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, f1_score


# Device

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)


# Dataset class

class CNCSequenceDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]


# Train / Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X_seq, y_seq, test_size=0.2, random_state=42, stratify=y_seq
)
```

```python
# LSTM Model

class LSTMModel(nn.Module):
    def __init__(self, input_dim, hidden_dim=64, num_layers=2):
        super().__init__()
        self.lstm = nn.LSTM(
            input_dim,
            hidden_dim,
            num_layers=num_layers,
            batch_first=True
        )
        self.fc = nn.Linear(hidden_dim, 2)

    def forward(self, x):
        _, (hn, _) = self.lstm(x)
        out = self.fc(hn[-1])
        return out

model = LSTMModel(input_dim=X_seq.shape[2]).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)


# Training Loop

EPOCHS = 10

for epoch in range(EPOCHS):
    model.train()
    total_loss = 0

    for Xb, yb in train_loader:
        Xb, yb = Xb.to(device), yb.to(device)

        optimizer.zero_grad()
        preds = model(Xb)
```

```python
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        print(f"Epoch [{epoch+1}/{EPOCHS}] - Loss: {total_loss/len(train_loader):.4f}")


# Evaluation

model.eval()
y_true, y_pred = [], []

with torch.no_grad():
    for Xb, yb in test_loader:
        Xb = Xb.to(device)
        preds = model(Xb)
        y_true.extend(yb.numpy())
        y_pred.extend(torch.argmax(preds, dim=1).cpu().numpy())

print("\n===== LSTM CLASSIFICATION REPORT =====")
print(classification_report(y_true, y_pred))
print("F1 Score:", f1_score(y_true, y_pred))
```

```
Using device: cpu
Epoch [1/10] - Loss: 0.6888
Epoch [2/10] - Loss: 0.6817
Epoch [3/10] - Loss: 0.6751
Epoch [4/10] - Loss: 0.6624
Epoch [5/10] - Loss: 0.6430
Epoch [6/10] - Loss: 0.5998
Epoch [7/10] - Loss: 0.5591
Epoch [8/10] - Loss: 0.5086
Epoch [9/10] - Loss: 0.4482
Epoch [10/10] - Loss: 0.4042
```

[redacted]

CNN MODEL (Temporal CNN on Sensor Signals)

```
[ ]
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, f1_score


# Dataset

class CNCDatasetCNN(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        # CNN expects (channels, time)
        return self.X[idx].transpose(0, 1), self.y[idx]


# Train / Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X_seq, y_seq, test_size=0.2, random_state=42, stratify=y_seq
)

train_ds = CNCDatasetCNN(X_train, y_train)
test_ds = CNCDatasetCNN(X_test, y_test)
```

[redacted]

CNN–LSTM HYBRID MODEL

```
[ ]
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, f1_score


# Dataset

class CNCDatasetHybrid(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]


# Train / Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X_seq, y_seq, test_size=0.2, random_state=42, stratify=y_seq
)

train_ds = CNCDatasetHybrid(X_train, y_train)
test_ds = CNCDatasetHybrid(X_test, y_test)

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=32)
```

[redacted]

MLP IMPLEMENTATION

```python
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    classification_report, f1_score,
    confusion_matrix, roc_curve, auc
)
import matplotlib.pyplot as plt
import numpy as np


#  DATASET

class CNCTabularDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.long)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]


#  SPLIT

X_train, X_test, y_train, y_test = train_test_split(
    X_tab_scaled, y, test_size=0.2, random_state=42, stratify=y
)
```

[redacted]

SHAP EXPLAINABILITY

```python
# !pip install shap
```

```python
import shap
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt


# Define Tabular MLP Model

class TabularMLP(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 16),
            nn.ReLU(),
            nn.Linear(16, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.net(x)


# Prepare Data

X_tab_np = X_tabular.astype(np.float32)
```

[redacted]

[redacted]

[redacted]

```
Epoch [1/20] Loss: 0.6629 Accuracy: 0.6429
Epoch [2/20] Loss: 0.6608 Accuracy: 0.6429
Epoch [3/20] Loss: 0.6587 Accuracy: 0.6429
Epoch [4/20] Loss: 0.6569 Accuracy: 0.6429
Epoch [5/20] Loss: 0.6553 Accuracy: 0.6429
Epoch [6/20] Loss: 0.6537 Accuracy: 0.6429
Epoch [7/20] Loss: 0.6523 Accuracy: 0.6429
Epoch [8/20] Loss: 0.6508 Accuracy: 0.6429
Epoch [9/20] Loss: 0.6495 Accuracy: 0.6429
Epoch [10/20] Loss: 0.6481 Accuracy: 0.6429
Epoch [11/20] Loss: 0.6469 Accuracy: 0.6429
Epoch [12/20] Loss: 0.6457 Accuracy: 0.6429
Epoch [13/20] Loss: 0.6446 Accuracy: 0.6429
Epoch [14/20] Loss: 0.6436 Accuracy: 0.6429
Epoch [15/20] Loss: 0.6426 Accuracy: 0.6429
Epoch [16/20] Loss: 0.6417 Accuracy: 0.6429
Epoch [17/20] Loss: 0.6409 Accuracy: 0.6429
Epoch [18/20] Loss: 0.6401 Accuracy: 0.6429
Epoch [19/20] Loss: 0.6393 Accuracy: 0.6429
Epoch [20/20] Loss: 0.6386 Accuracy: 0.6429

===== MLP CLASSIFICATION REPORT =====
              precision    recall  f1-score   support

           0       0.00      0.00      0.00         1
           1       0.75      1.00      0.86         3

    accuracy                           0.75         4
   macro avg       0.38      0.50      0.43         4
weighted avg       0.56      0.75      0.64         4
```

```
===== LSTM CLASSIFICATION REPORT =====
              precision    recall  f1-score   support

           0       0.70      0.45      0.55        42
           1       0.67      0.85      0.75        54

    accuracy                           0.68        96
   macro avg       0.69      0.65      0.65        96
weighted avg       0.68      0.68      0.66        96

F1 Score: 0.7479674796747967
```

```
===== CNN CLASSIFICATION REPORT =====
              precision    recall  f1-score   support

           0       0.73      0.71      0.72        42
           1       0.78      0.80      0.79        54

    accuracy                           0.76        96
   macro avg       0.76      0.76      0.76        96
weighted avg       0.76      0.76      0.76        96

F1 Score: 0.7889908256880734
```

[redacted]

[redacted]

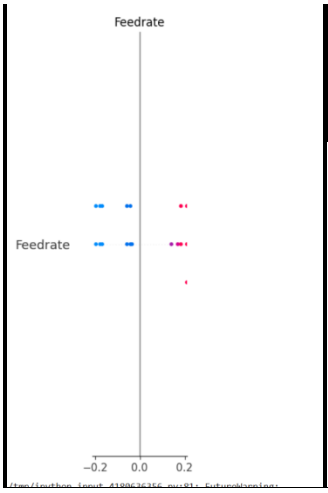
[redacted]

```
===== CNN-LSTM CLASSIFICATION REPORT =====
              precision    recall  f1-score   support

           0       0.70      0.38      0.49        42
           1       0.64      0.87      0.74        54

    accuracy                           0.66        96
   macro avg       0.67      0.63      0.62        96
weighted avg       0.67      0.66      0.63        96

F1 Score: 0.7401574803149606
```

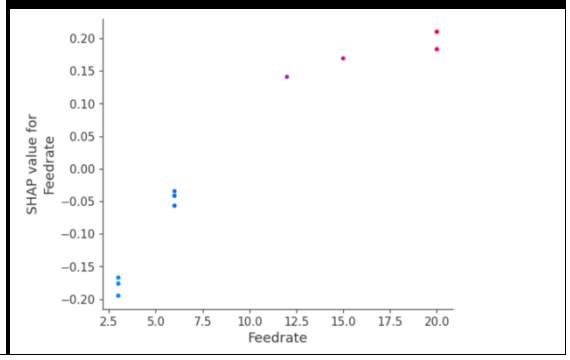
[redacted]

[redacted]

[redacted]



[redacted]

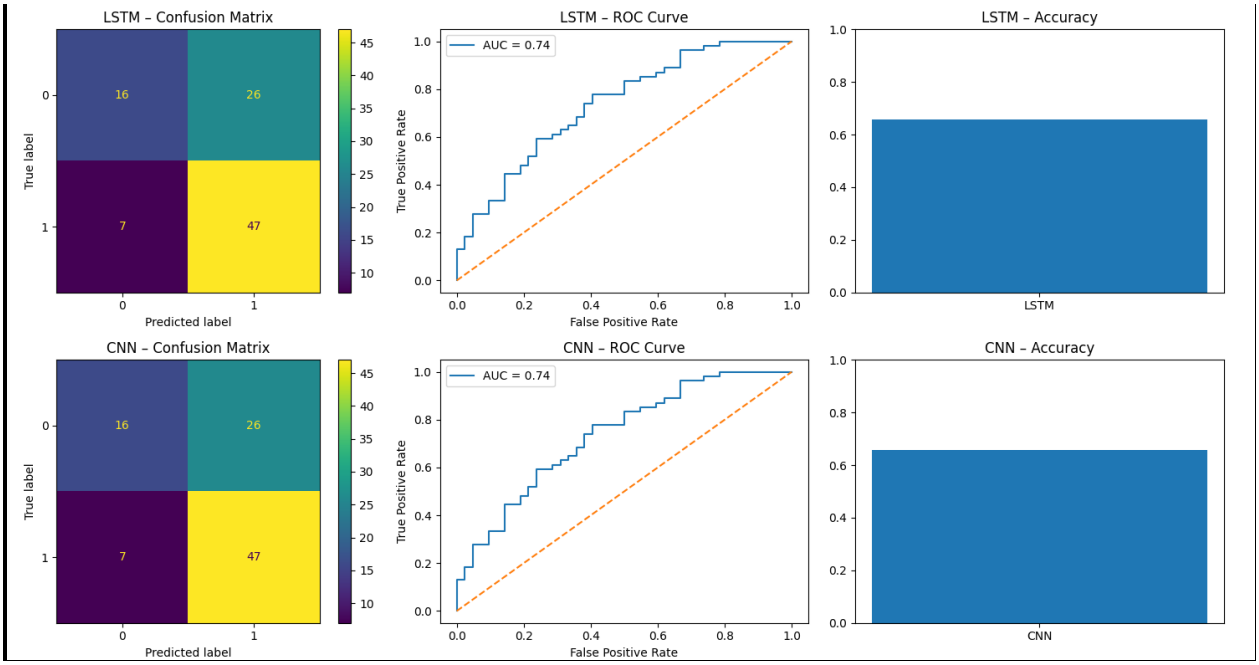


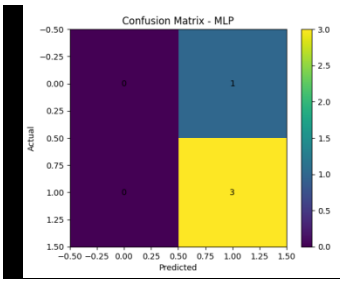
[redacted]

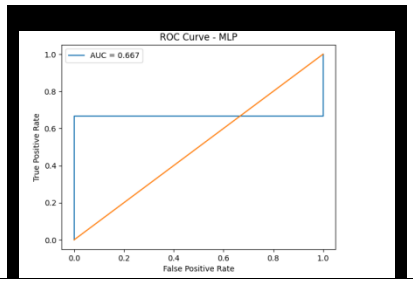
[redacted]

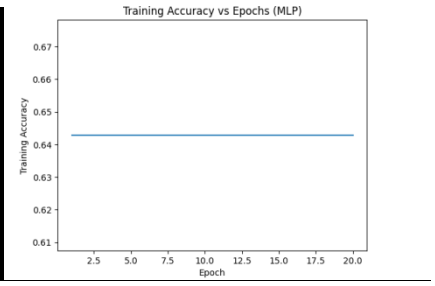
[redacted]

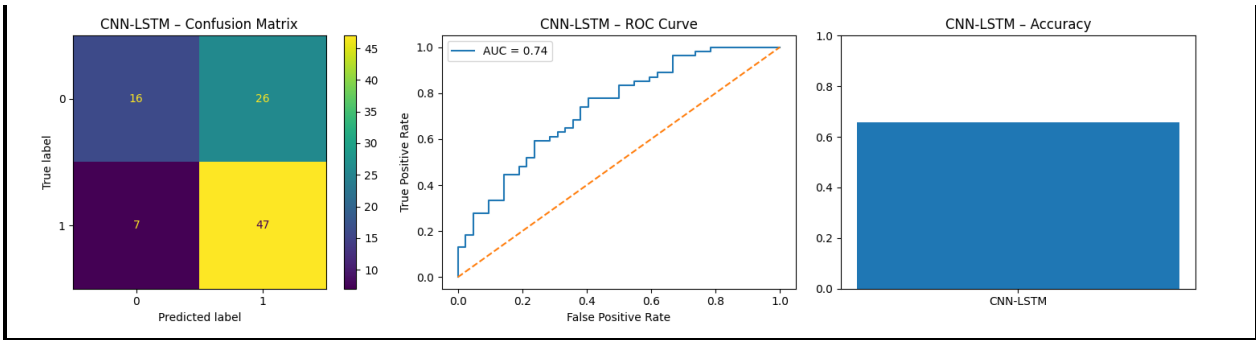
[redacted]











[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]